

K-Nearest Neighbors (KNN) Algoritması

K-En Yakın Komşu (KNN) algoritması, makine öğrenmesinde hem sınıflandırma hem de regresyon problemleri için kullanılan basit ama etkili bir algoritmadır. Özellikle anlaması ve uygulaması kolay olduğu için öğretici amaçlarla sıkça tercih edilir.

KNN ile Sınıflandırma

KNN'nin temel fikri çok basittir: **Bir veri noktasının sınıfını, ona en yakın komşularının çoğunluğuna bakarak belirler.**

- 1- Komşu sayısı (bir k değeri) belirlenir.
- 2-Yeni gelen test verisi için eğitim veri setindeki tüm örneklerle mesafe hesaplanır.
- 3-Mesafesi en yakın k örnek seçilir.
- 4-Bu k komsunun sınıf etiketleri sayılarak en sık gorulen sınıf etiketi tahmin edilir.

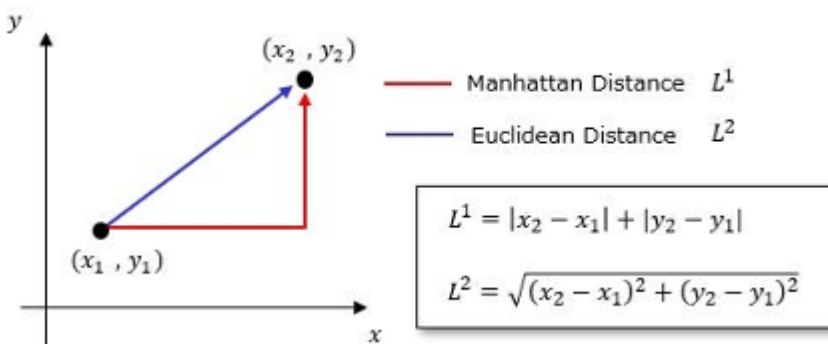
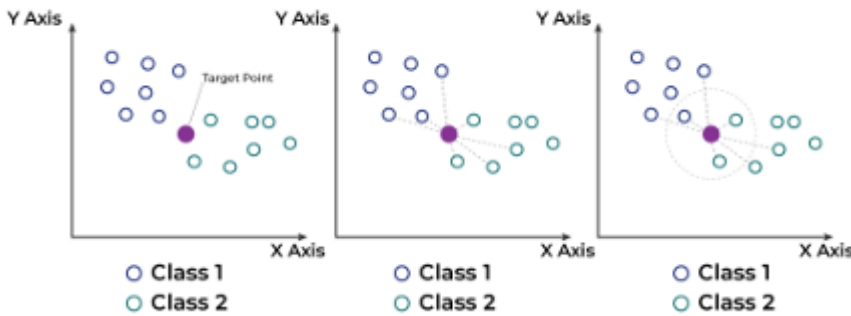
Example;

İki boyutlu bir uzayda, sınıfı 0 ve 1 olan veriler verilmiş olsun. Yeni bir test noktasının konumu verildiğinde, mesafesi en yakın 5 komsuya (k = 5) bakılır. Bu 5 komsunun çoğunluğu sınıf 1 ise, test noktasının sınıfı da 1 olarak atanır.

Mesafe Ölçümleri

KNN algoritması, test noktasına olan mesafeyi ölçerek en yakın komşuları bulur. En yaygın kullanılan iki mesafe ölçüsü şunlardır:

İşte bu seçilen örneklerden hangisi fazlaysa seçer onu.



KNN'in Önemli Özellikleri

- **Tembel Öğrenme (Lazy Learning):** KNN, eğitim aşamasında bir model oluşturmaz. Sadece tüm eğitim verisini depolar. Tahmin yapması gerektiğinde, yani yeni bir veri geldiğinde hesaplamalara başlar. Bu yüzden "tembel" olarak adlandırılır.
- **Mesafe Metriği:** En yakın komşuları bulmak için veri noktaları arasındaki mesafeyi hesaplar. En yaygın kullanılan mesafe metrikleri **Öklid mesafesi** veya **Manhattan mesafesidir**. Bu mesafe hesaplamaları nedeniyle, KNN için veri setini **StandardScaler** gibi yöntemlerle ölçeklemek çok önemlidir. Aksi takdirde, büyük değerlere sahip özellikler (örneğin sıcaklık), küçük değerlere sahip özelliklere (örneğin nem) göre mesafeyi çok daha fazla etkiler.
- **K Değeri:** Seçilen K değeri, modelin performansını doğrudan etkiler.
 - **Küçük K değeri:** Gürültüye duyarlıdır ve overfitting'e yol açabilir.
 - **Büyük K değeri:** Modelin karmaşıklığını azaltır ve daha genelleştirilebilir sonuçlar verir, ancak modelin tahminleri daha az doğru olabilir. En iyi K değerini bulmak için genellikle **GridSearchCV** gibi yöntemler kullanılır.

KNN ile Sınıflandırma : Uygulamalı Örnek

İki özelliğe sahip 6 gözlem noktamız olduğunu düşünelim :Boy(height) ve Kilo (weight). Ayrıca her bireyin sporcu (athlete) olup olmadığı bilgisi (etiket) verilmiştir:

ID	Boy (cm)	Kilo (kg)	Athlete (Label)
1	170	65	Evet
2	165	72	Hayır
3	180	80	Evet
4	175	85	Hayır
5	160	60	Hayır
6	172	70	Evet

Yeni bir kişi için→
boy=168 cm, kilo = 66 kg

Mesafeleri Öklidyen formulu ile hesaplayalım;

$$\text{Mesafe} = \sqrt{(x_{\text{boy}} - x'_{\text{boy}})^2 + (x_{\text{kilo}} - x'_{\text{kilo}})^2}$$

Gözlem	Mesafe (Yeni kişiye)
1	$\sqrt{(170 - 168)^2 + (65 - 66)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.24$
2	$\sqrt{(165 - 168)^2 + (72 - 66)^2} = \sqrt{9 + 36} = \sqrt{45} \approx 6.70$
3	$\sqrt{(180 - 168)^2 + (80 - 66)^2} = \sqrt{144 + 196} = \sqrt{340} \approx 18.44$
4	$\sqrt{(175 - 168)^2 + (85 - 66)^2} = \sqrt{49 + 361} = \sqrt{410} \approx 20.25$
5	$\sqrt{(160 - 168)^2 + (60 - 66)^2} = \sqrt{64 + 36} = \sqrt{100} = 10.00$
6	$\sqrt{(172 - 168)^2 + (70 - 66)^2} = \sqrt{16 + 16} = \sqrt{32} \approx 5.66$

En Yakın 3 Komşunun Belirlenmesi (k=3);

Gözlem 1 (2.24) → Etiket : Evet

Gözlem 6 (5.66) → Etiket : Evet

Gözlem 2 (6.70) → Etiket : Hayır

Sınıf Etiketinin Belirlenmesi

2 Evet, 1 hayır olduğu için sonuç → Evet, Athlete

Bu duruma (voting) oylama denir.

KNN ile Regresyon

Sınıflandırma yönteminde en yakın komşular bulunup en çok komşunun hangi sınıfta olduğunu bulurken, regresyon yönteminde en yakın komşuların ortalaması alınarak tahmin edilen değer bulunur.

Test verisinin k komşusu belirlendikten sonra, bu k komşunun hedef değişken değerlerinin ortalaması alınır:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^k y_i$$

Not: Eğer veri setinde uç değerler varsa, ortalama yerine medyan kullanımı daha iyi sonuç verebilir.

KNN Optimizasyonu : KD-Tree ve Ball-Tree

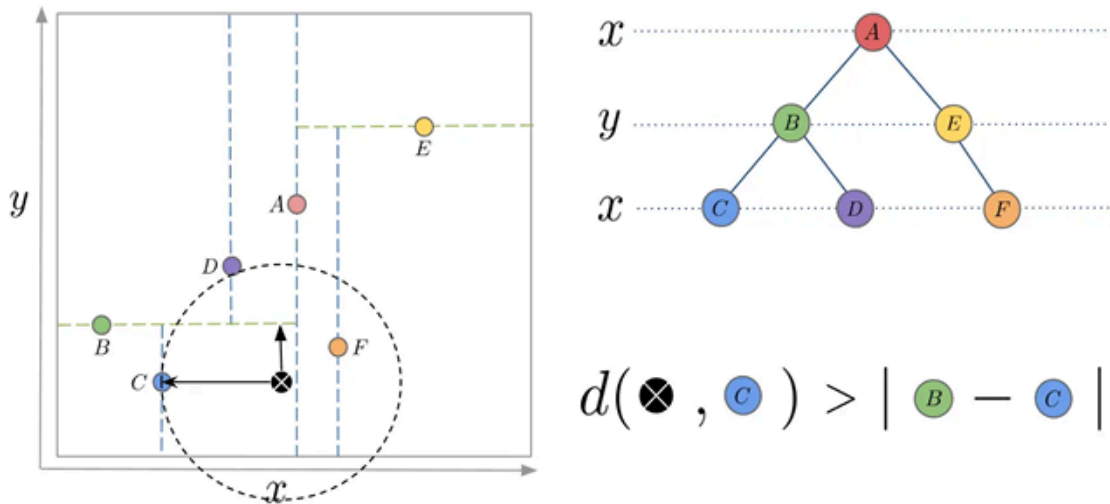
KNN algoritmasında test verisi için tüm eğitim verileriyle mesafe hesaplanması gerekir. Bu işlem büyük veri setlerinde zaman açısından oldukça maliyetlidir (**O(n)** zaman karmaşıklığı) ki bu direkt **Brute-Force**'dir Bu sorunu azaltmak için iki veri yapısı kullanılabilir.

KD-Tree (k-dimensional tree)

Nasıl Çalışır? Kd-Tree, verileri art arda bölgelere bölerek bir ağaç yapısı oluşturur. Her bir seviyede, bir boyut (örneğin, x eksenini, sonra y eksenini vb.) seçer ve verileri bu boyuttaki medyan değere göre ikiye ayırır. Bu işlem, her alt grup için tekrarlanır.

Avantajı: Veri boyutunun (feature) az olduğu durumlarda çok etkilidir. En yakın komşu aramasını, verinin sadece küçük bir bölümünde yoğunlaştırarak hızlandırır.

Dezavantajı: Veri boyutu çok yüksek olduğunda (örneğin 20'den fazla özellik), bu yapının performansı düşer. Bunun nedeni, yüksek boyutlu uzayda verilerin seyrelmesi ve ağaç yapısının verimsiz hale gelmesidir.



Her sorguda yalnızca belirli bölgelerde arama yapıldığı için arama süresi önemli ölçüde azalır.

- İlk başta x'in medyanı A noktası olsun. Yeni veri A'ya göre hangi taraftaysa işte öyle öyle medyan çizerek küçük küçük bölgelere sıkıştırıyorsun orda arıyorsun.
- Tabi dikey ve yatay sırada bölüyorsun.
- Hangi kümeye düşerse sadece o kümedeki noktalarla işini yapıyorsun.

Ball Tree

Nasıl Çalışır? Ball Tree, verileri dikdörtgenler yerine **içi boş top veya kürelerle** (ball) gruplandırarak bir ağaç yapısı oluşturur. Her bir düğüm, bir topu temsil eder ve bu top, içerdiği tüm veri noktalarını çevreler.

Avantajı: Yüksek boyutlu veri setleri için Kd-Tree'den daha etkilidir. Veri dağılımı rastgele veya düzensiz olduğunda daha iyi performans gösterir.

Dezavantajı: Oluşturulması ve yönetilmesi Kd-Tree'ye göre daha karmaşıktır.

